# 0073A- Compilation Techniques

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

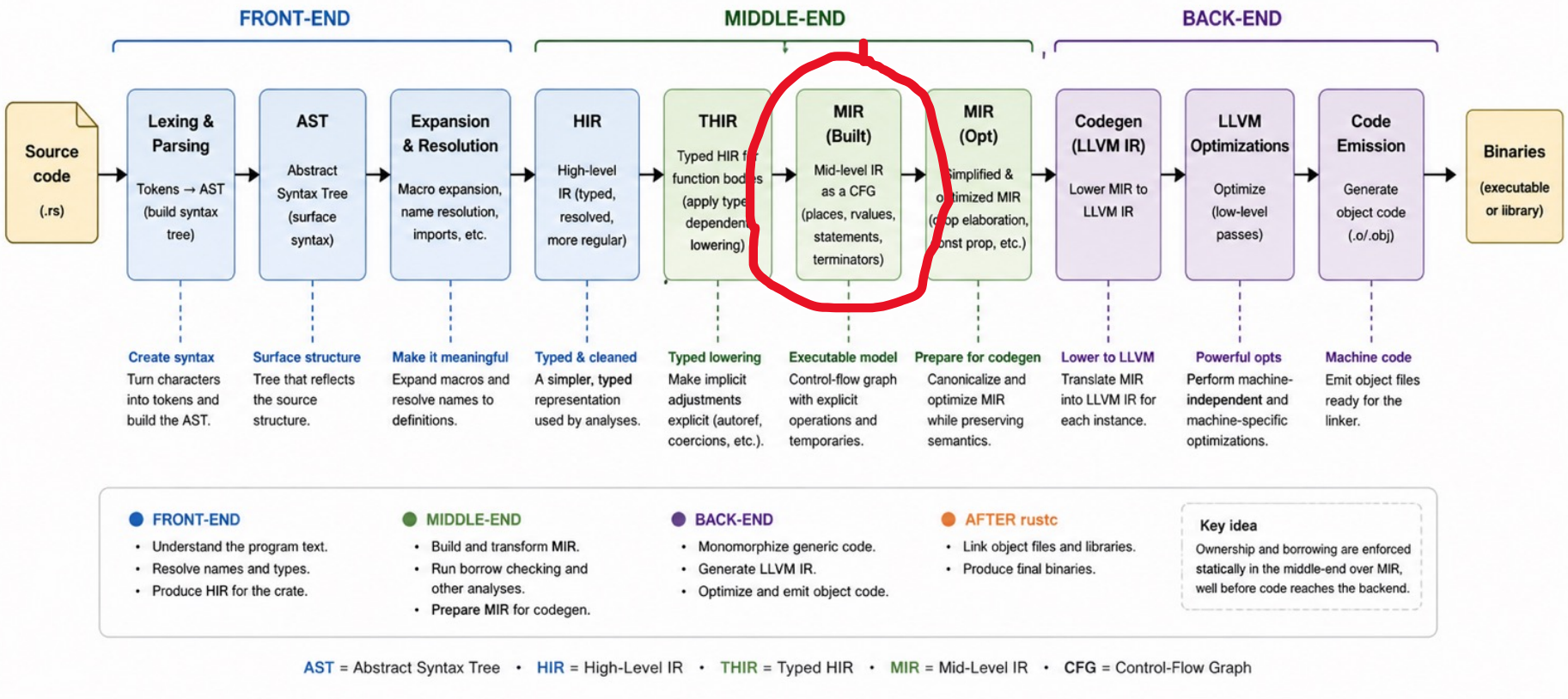
http://pages.di.unipi.it/corradini/

***CompTech-26**: RUST #4*

# Aspects of Rust Compilation (4)

- Region inference during Borrow checking
- Constraint generation
- Graph of outlives constraints and SCC's
- Constraint propagation on the DAG of SCC's
- Type tests and Universal Regions checking
- Two-phase borrows
- Closure Capture Inference

# The rustc Compilation Pipeline



# Borrow Checking as Program Analysis

- **Domain**: The checker reasons about places, move paths, loans, initialized states, and regions at locations in a MIR CFG.
- **Constraints**: Type checking over MIR generates region constraints, while move and borrow analyses produce facts over program points.
- **Dataflow**: Classic gen/kill and fixed-point ideas appear in the computation of initialization, liveness, and loan validity.
- **Diagnostics**: The result is not only accept or reject; the compiler maps facts back to source spans to explain why a borrow or move is invalid.

# Major Phases of the MIR Borrow Checker

- **Entry point:** mir_borrowck query
  The borrow checker is implemented in rustc_borrowck and is invoked as a query over MIR.
- **MIR duplication and preparation**
  A local copy of MIR is created and will be **mutated in-place** to attach region information.
- **Region variable initialization**
  replace_regions_in_mir replaces all lifetime annotations with **fresh inference variables**.
- **Dataflow analyses (moves and initialization)**
  Analyses compute **when values are moved, initialized, or invalidated** across the CFG.
- **Second type checking over MIR**
  A MIR-level type check derives **constraints between regions (lifetimes)**.
- **Region inference (constraint solving)**
  Computes the **set of CFG points where each lifetime must hold**, via fixpoint-style inference.
- **Borrow sets / borrows in scope**
  Determines which borrows are **active at each program point** in the control-flow graph.
- **Final validation pass (error reporting)**
  A second traversal checks operations (e.g., *a + 1) against:
  - initialization state
  - borrowing rules
  - mutability constraints

# The MIR type check

- A key component of the borrow check is the MIR type-check. This check walks the MIR and does a complete "type check".
- During this type-check, it also uncover region constraints that apply to the program, as well as **type tests** (special obligations about types involving regions)
- It replaces all regions in the body with new unconstrained regions.
- Even though MIR is built from THIR—which is already fully type checked—rustc still performs a MIR type check because the lowering process is not type-preserving in a trivial way and introduces new constructs and invariants that must be validated independently.

# Region Inference (NLL) — Overview

- Step 1: Identify universal regions
  - Extract free regions from function signature (e.g., `'a, 'static`)
  - Represent caller-visible lifetimes
- Step 2: Replace regions with variables
  - All MIR regions → fresh inference variables
  - Discards lexical lifetime info → enables control-flow-sensitive (NLL) analysis
- Step 3: Generate constraints (MIR type check)
  - Specialized MIR type checker produces:
    - outlives constraints (`'a:'b`)
    - liveness constraints
    - type tests (special obligations about types involving regions)
- Step 4: Solve constraints (region inference)
  - Build RegionInferenceContext
  - Compute region values via constraint propagation / fixpoint solving
- Error checking after inference:
  - Check type tests
  - Universal region check (“too big” regions)

# Constraints

Before region inference, constraints are collected by the MIR type checker.

Kind of constraints:

- liveness constraints (R live at E);
- outlives constraints (R1: R2), which arise from subtyping;
- member constraints (member R_m of [R_c...]), which arise from impl Trait.

The MIR type checker also collects type tests, to be checked **after** region inference.

# Examples of generated constraints

```
fn f<'a>(x: &'a u32) -> u32
{       let y = x;
        *y  }
```

- Simple liveness constraint
- The borrow/reference stored in **`y`** is **live** at the point where **`*y`** is used.
- Outlives constraint from assignment
  - The compiler introduces an inferred region, say '0, for y:
  - For the assignment to be valid:
  - Which implies 'a : '0

```
x : &'a u32
y : &'0 u32
```

```
&'a u32 <: &'0 u32
```

# Region Inference (NLL) — How It Works

- Initialization from liveness
  - Each region starts with CFG points where it is used
  - These come from liveness constraints
- Constraint propagation
  - For each outlives constraint **`'a:'b`**:

    **`'a ← 'a U 'b U {end('b)}`**
  - Repeated until fixpoint (propagate_constraints)
- Regions grow monotonically
  - Region values are sets of elements
  - Propagation = union-based dataflow

# Cycles of outlives constraints

- Outlives constraints ('a: 'b) are represented as a **directed graph** where regions are nodes and constraints are edges.
- Cycles in this graph indicate regions that must be **equal**, so the compiler computes **strongly connected components (SCCs)** to group such regions together.
- Each SCC is treated as a single unit with one shared value, improving efficiency.
- After collapsing cycles into SCCs, the compiler works on a **reduced graph of SCCs**, which is always a **DAG**.
- Constraints between regions become constraints between SCCs, enabling efficient propagation:  potentially cyclic dependencies are avoided.

# Type tests

During MIR type checking, rustc generates constraints and type tests, special obligations about types involving regions.

The region inference solver computes a solution (a mapping from region variables to sets of program points).

After the solution is computed, rustc checks all the stored type tests against that solution.

They are used when:

- encoding a constraint directly in the solver would be too complex or inefficient
- or when it is cleaner to check it *after* we know the final regions

# What do they check?

A type test typically verifies that **a type is well-formed with respect to the inferred lifetimes**.

Examples (conceptually):

- A reference type `&'r T` is valid only if `'r` contains all all points where the reference is used
- A generic constraint like `T:'a` requires that all references inside `T` live enough to satisfy `'a`
- Subtyping relations between types involving lifetimes are respected

So they are essentially well-formedness / subtyping checks on types that depend on regions.

# Why not encode them in the solver?

Because the solver works on a relatively simple domain (sets of points, outlives constraints, etc.), while:

- type structure can be complex (nested types, projections, generics)
- embedding all that into the fixpoint computation would complicate and slow it down

| Rust code | Generated type test | Short explanation |
|---|---|---|
| `fn f<'a, T>(x: &'a T) {`<br>`  let y: &'a T = x;`<br>`}` | `T: 'a` | The type `&'a T` is well-formed only if the referent type `T` is valid for at least `'a`.<br>➔ Satisfied |
| `fn f<'a, T>(x: T) {`<br>`  let r: &'a T;`<br>`}` | `T: 'a` | Even if `r` is only a local declaration, the annotated type `&'a T` imposes a well-formedness obligation on `T`.<br>➔ Not satisfied |
| `fn f<'a, T>(x: &'a [T])`<br>`{`<br>`  let y: &'a [T] = x;`<br>`}` | `[T]: 'a,`<br>reduced/checkable as `T: 'a` | A slice reference with lifetime `'a` requires the slice element type to be valid for `'a`.<br>➔ Satisfied |
| `struct S<T>(T);`<br><br>`fn f<'a, T>(x: &'a S<T>)`<br>`{}` | `S<T>: 'a,`<br>which requires `T: 'a` | The reference `&'a S<T>` requires the whole referent type `S<T>` to outlive `'a`; structurally this depends on `T`. |

# Simple type tests from return type

```
fn f<'a,'b>(x: &'a u32) -> &'b u32 {
  x }
```

- Returning **`x`** where **`&'b u32`** is expected requires: `&'a u32 <: &'b u32`
- so the compiler generates the following type test: `'a: 'b`
- This is not used as an outlives constraint, but it is only checked to hold after region inference
- Thus compilation fails

# Universal Regions and Lifetime Inference

- *Universal regions* refer to user-declared lifetimes such as lifetime parameters (`'a`, `'b`) and `'static`.
- They are universally quantified: the compiler must ensure the function is correct for *all possible instantiations* of these lifetimes (satisfying the declared constraints)
- Internally, all lifetimes are represented as region variables:
  - universal regions occupy a fixed subset of these variables
  - existential regions represent inference variables
- During region inference, each region is assigned a set containing control-flow points and special markers like `end('a)` indicating the extent of a lifetime.
- After propagation, the compiler performs a **consistency check**: if the inferred value of a universal region `'a` includes `end('b)`, then the relationship `'a:'b` must have been explicitly declared.
- If not, this indicates a violation of the function's signature and results in a compile-time error.
- This ensures that inferred lifetime relationships do not exceed those promised by the function's bounds.

# ~~Shared~~ Dataflow Structure

| Aspect | Move Analysis | Region Inference (NLL) |
|---|---|---|
| Domain | Set of move paths | Set of region elements |
| Direction | Forward | Forward (propagation) |
| Lattice | Powerset | Powerset |
| Transfer | GEN/KILL | Union via constraints |
| Meaning | “is initialized?” | “is region valid here?” |
| Fixpoint | Yes | Yes |

- Move analysis: on the MIR CFG
- Region inference: on the DAG of SCC of the graph (Regions, Outlives)
- Borrow sets / borrows in scope
  Determines which borrows are active at each program point in the control-flow graph.
- Final validation pass (error reporting)
  A second traversal checks operations (e.g., *a + 1) against:
  - initialization state
  - borrowing rules
  - mutability constraints

# Liveness ends at last use: NLL example

- The shared borrow **`r`** is live only until its last use:

```
'r live at println!("{}", r)
```

```
fn f() {
    let mut x = 10;
    let r = &x;
    println!("{}", r);
    let m = &mut x;
    *m += 1;
}
```

- After that point, the region of **`r`** ends, so the mutable borrow can begin.
- Conceptually:

```
'r = { points up to last use of r }
'm = { points from &mut x to *m += 1 }
```

  - Because these regions do not overlap, the program is accepted.

# Rejected because liveness overlaps with mutable borrow

- Here `r` is still live at the `println!` after `m` is created.
- So the compiler sees:

```
fn f() {
    let mut x = 10;
    let r = &x;
    let m = &mut x;
    println!("{}", r);
    *m += 1;
}
```

```
'r live at println!("{}", r)
'm live at *m += 1
```

- and the shared borrow and mutable borrow overlap. This violates the borrowing rules, so the program is rejected.

# Two-phase borrows

- Special form of mutable borrow that temporarily behave like shared borrows to allow patterns such as `vec.push(vec.len())`.
- Introduced only in specific implicit cases (e.g., method calls with **`&mut`** receivers, reborrows in arguments, and compound assignment operators)
- Implemented as temporaries with two key points:
  - a **reservation point**, where the shared-like borrow begins
  - an **activation point**, where it becomes a full mutable borrow.
- Treated like mutable borrows but with relaxed rules before activation.

# Two-phase borrows (2)

```
fn push_len(v: &mut Vec<i32>) {
    v.push(v.len() as i32); }
```

```
// Conceptual THIR, simplified
Vec::push(&mut *v, (Vec::len(&*v)) as i32)

// MIR

fn push_len(_1: &mut Vec<i32>) -> () {
    let mut _0: ();
    let mut _2: &mut Vec<i32>;
    let mut _3: &Vec<i32>;
    let mut _4: usize;
    let mut _5: i32;

    bb0: {
[p1]    _2 = &mut (*_1); // reservation of mutable borrow
[p2]    _3 = &(*_1);     // shared borrow for len()
[p3]    _4 = Vec::<i32>::len(move _3);
[p4]    _5 = move _4 as i32 (IntToInt);
[p5]    _0 = Vec::<i32>::push(move _2, move _5);
        return;
    }
}
```

Fresh regions:

```
_2: &'r_mut mut Vec<i32>
_3: &'r_shr Vec<i32>
```

Region solution:

```
'r_shr = { p2, p3 }
'r_mut = { p1, p2, p3, p4, p5 }
```

But `'r_mut` is **two-phase**:

```
reservation: p1
activation: p5
```

So, between `p1` and `p5` the mutable borrow is only reserved, and the use of the shared borrow at `p3` is legal

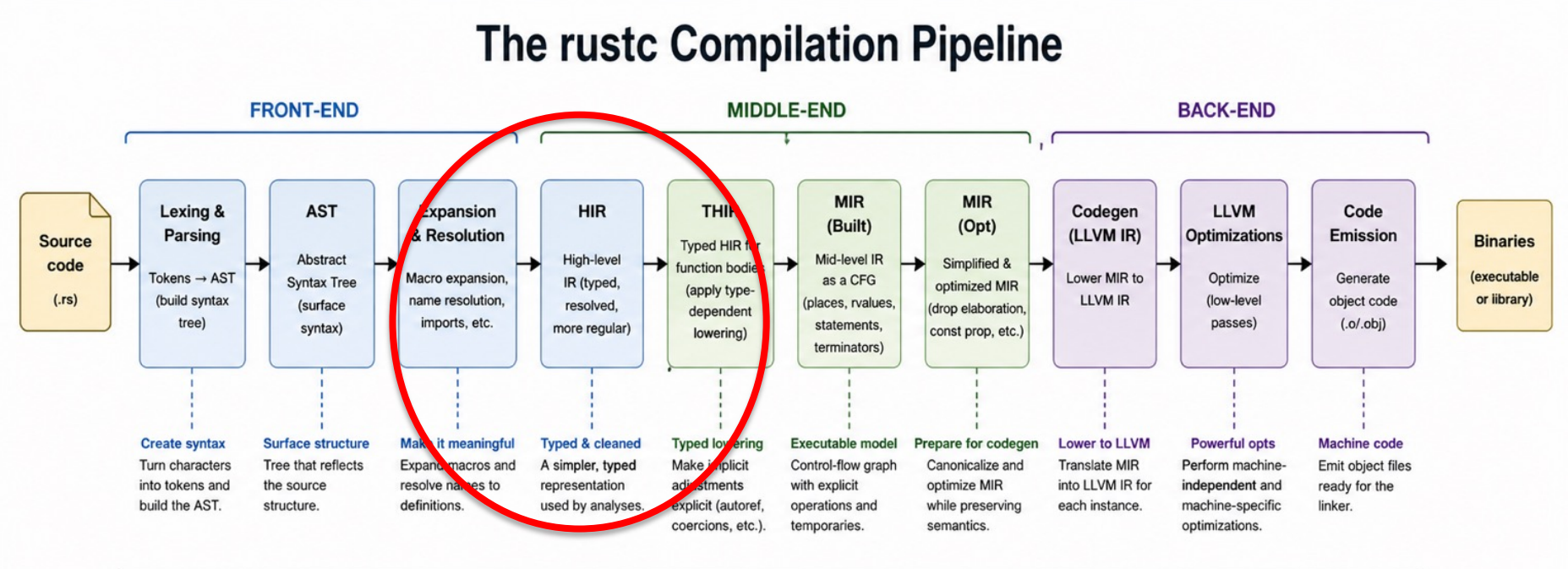
# Closure Capture Inference

Rust closures are compiled by transforming them into structs that store captured variables. The compiler (rustc) must:

- Identify which variables from the surrounding scope the closure uses
- Determine how each is used (read, modified, or consumed)
- Based on that, decide the capture mode:
  - by shared reference (&T)
  - by mutable reference (&mut T)
  - by value (move)

This analysis also lets rustc infer which closure trait the closure implements:

- Fn (only reads)
- FnMut (mutates)
- FnOnce (consumes values)




- Closure inference in rustc is performed during the **type checking phase**, specifically within the **HIR (High-level Intermediate Representation) type-checking stage**.

More precisely:

- It happens after parsing and lowering to HIR
- During **type inference and borrow checking preparation**

# Recall: Closures are desugared

| Source | Desugared |
|---|---|
| `let f = \|x\| x + 1;`<br>`// closure` | `struct Closure;`<br>`impl Fn(i32) -> i32 for Closure {`<br>`  extern "rust-call" fn call(&self, (x,): (i32,))`<br>`-> i32 {`<br>`  x + 1`<br>`  }}`<br>`let f = Closure;` |
| `let y = 10;`<br>`let f = \|x\| x + y;`<br>`// variable capturing` | `struct Closure { y: i32 }`<br>`impl Fn(i32) -> i32 for Closure {`<br>`  extern "rust-call" fn call(&self, (x,): (i32,))`<br>`-> i32 {`<br>`  x + self.y}}`<br>`let f = Closure { y };` |

rustc internal desugaring:

- The resulting syntax is not legal Rust code
- Capture inference and trait selection is not expressible directly by user-written code

# Variable capture – read only

```
fn invoke(f: impl Fn()) {     // also FnMut(), FnOnce()
    f();
}

fn main() {
    let x: i32 = 10;
    invoke(|| println!("Hi {}", x));  // The closure just reads x.
    println!("Value of x after return {}", x);
}
```

Generate the MIR with

```
rustc --emit=mir immut.rs
```

```
bb0: {
        _1 = const 10_i32;
        _4 = &_1;
        _3 = {closure@immut.rs:13:12: 13:14} { x: move _4 }; // struct with init
        _2 = invoke::<{closure@immut.rs:13:12: 13:14}>(move _3) -> [return: bb1,
unwind continue];
    }
```

# Variable capture – mutate

```
fn invoke(mut f: impl FnMut()) {  // also FnOnce()
    f();
}

fn main() {
    let mut x: i32 = 10;
    invoke(|| {
        x += 10;  // The closure mutates the value of x
        println!("Hi {}", x)
    });
    println!("Value of x after return {}", x);
}
```

rustc --emit=mir mut.rs

```
bb0: {
        _1 = const 10_i32;
        _4 = &mut _1;
        _3 = {closure@mut.rs:7:12: 7:14} { x: move _4 }; // struct with init
        _2 = invoke::<{closure@mut.rs:7:12: 7:14}>(move _3) -> [return: bb1, unwind
continue];
    }
```

# Variable capture – move

```
fn invoke(f: impl FnOnce()) { // errore con Fn() o FnMove()
    f();
}

fn main() {
    let x = vec![21];
    invoke(|| {
        drop(x);  // Makes x unusable after the fact.
    });
    // println!("Value of x after return {:?}", x);
}
```

```
rustc --emit=mir drop.rs
```

```
bb2: {
        _4 = {closure@drop.rs:7:12: 7:14} { x: move _1 };  // move di x
        _3 = invoke::<{closure@drop.rs:7:12: 7:14}>(move _4) -> [return: bb3, unwind
continue];
    }
```

# Inferences in the compiler

- An **upvar** is a variable from the surrounding function that a closure captures (a "free variable").
- The compiler identifies these using an internal analysis (upvars_mentioned).
- Closures differ from normal functions because they capture and borrow these upvars from their environment.
- rustc infers how each upvar is captured by starting with an immutable borrow and relaxing it if needed:
  - stays immutable if only read (as in Example 1)
  - becomes mutable if modified (as in Example 2)
  - becomes move if consumed (e.g., dropped) (as in Example 3)
- Based on this, the closure implements the appropriate trait:
  - Fn → immutable borrow
  - FnMut → mutable borrow
  - FnOnce → move semantics

# Inferring kind of variable capture using a visitor

```
fn main() {
    let mut x = vec![21];
    let _cl = || {
        let y = x[0];  // 1.
        x[0] += 1;  // 2.
    };
}
```

- upvar.rs defines the euv::ExprUseVisitor which walks the source of the closure and invokes a callback for each upvar that is borrowed, mutated, or moved.
- In the above example, the visitor will be called twice, for the lines marked 1 and 2, once for a shared borrow and another one for a mutable borrow. It will also tell us what was borrowed.
- The callbacks are defined by implementing the Delegate trait.

- The compiler uses a callback system (Delegate trait) to analyze how closures use variables.
- InferBorrowKind implements this trait and tracks how each upvar is captured:
  - ByValue (moved)
  - ByRef (borrowed), with kinds: ImmBorrow, UniqueImmBorrow, MutBorrow
- Key callbacks:
  - consume → variable is moved
  - borrow → variable is borrowed
  - mutate → variable is modified
- Each callback receives a `cmt` (Category, Mutability, Type) describing the variable's origin, location, and mutability.
- Based on these callbacks, the compiler adjusts the borrow kind and records the final capture mode for each closure.

# Closure Inference: Summary

- Rust infers how closures capture variables through a static analysis of their usage.
- Variables from the surrounding scope, called *upvars*, are identified and initially assumed to be immutably borrowed.
- As the compiler analyzes how each upvar is used—whether it is read, mutated, or consumed—it progressively relaxes this assumption to mutable borrowing or moving if necessary.
- This process is implemented via a callback-based mechanism (Delegate), where operations like borrow, mutate, and consume update the inferred capture mode using contextual information (cmt).
- The compiler records the final capture kind for each upvar (by reference or by value, with specific borrow kinds), and from this determines which closure trait (Fn, FnMut, or FnOnce) the closure can implement.